C++ basic

- extension of c++ => .cpp

1. setup c++
2. print
3. input
4. type conversion
5. eof (end of file)

Conditional statements:

1️⃣ `if` Statement

🔹 What

একটা condition **true** হলে কোড execute হবে।

🔹 Syntax

```
if (condition) {
    // code
}
```

🔹 Example

```
int x = 10;

if (x > 5) {
    cout << "x is greater than 5";
}
```

🔹 When to use

- একটাই condition check করতে হলে
- Simple decision নিতে হলে

2️⃣ `if-else` Statement

🔹 What

Condition true হলে এক কাজ, না হলে অন্য কাজ।

🔹 Syntax

```
if (condition) {
    // true block
} else {
    // false block
}
```

🔹 Example

```
int age = 16;

if (age >= 18) {
    cout << "Adult";
} else {
    cout << "Not Adult";
}
```

🔹 Use-case

- Pass / Fail
- Login success / fail

3️⃣ `if - else if - else` (Multiple Conditions)

🔹 What

একাধিক condition check করা যায়।

🔹 Syntax

```
if (condition1) {
}
else if (condition2) {
}
else {
}
```

🔹 Example

```
int marks = 75;

if (marks >= 80) {
    cout << "A+";
```

```
}
else if (marks >= 70) {
    cout << "A";
}
else if (marks >= 60) {
    cout << "B";
}
else {
    cout << "Fail";
}
```

🔹 When to use

- Grading system
- Menu selection
- Range-based decision

4️⃣ Nested `if` (if এর ভিতরে if)

🔹 What

একটা `if` এর ভিতরে আরেকটা `if`

🔹 Example

```
int age = 20;
bool hasID = true;

if (age >= 18) {
    if (hasID) {
        cout << "Allowed";
    } else {
        cout << "ID Required";
    }
}
```

🔹 Use-case

- Multiple dependent conditions
- Security checks

5️⃣ `switch` Statement

🔹 What

একটা variable এর **multiple fixed values** এর উপর decision।

🔹 Syntax

```
switch (expression) {
    case value1:
        // code
        break;
    case value2:
        // code
        break;
    default:
        // code
}
```

🔹 Example

```
int day = 3;

switch (day) {
    case 1:
        cout << "Monday";
        break;
    case 2:
        cout << "Tuesday";
        break;
    case 3:
        cout << "Wednesday";
        break;
    default:
        cout << "Invalid Day";
}
```

🔹 Rules

- `break` না দিলে fall-through হবে
- Only **int, char, enum** allowed
- No condition like `>`, `<`

🔹 When to use

- Menu-driven program
- Options selection

6️⃣ Ternary Operator (`?:`)

🔹 What

`if-else` এর **short form**

🔹 Syntax

```
condition ? expression1 : expression2;
```

🔹 Example

```
int a = 10, b = 20;

int max = (a > b) ? a : b;
cout << max;
```

🔹 Use-case

- Small decisions
- One-line conditions

7️⃣ Conditional Operator with `cout`

```
int num = 7;

cout << (num % 2 == 0 ? "Even" : "Odd");
```

8️⃣ Logical Conditions in C++

Operator	Meaning		
&&	AND	&	bitwise AND
||	logical OR	|	bitwise OR
!	NOT		

🔹 Example

```
int age = 25;
bool citizen = true;

if (age >= 18 && citizen) {
    cout << "Eligible to vote";
}
```

9️⃣ Comparison Operators (Used in condition)

Operator	Meaning
`==`	Equal
`!=`	Not Equal
`>`	Greater
`<`	Less
`>=`	Greater or equal
`<=`	Less or equal

🔟 Real-World Mini Example

```
int balance = 5000;
int withdraw = 3000;

if (withdraw <= balance) {
    balance -= withdraw;
    cout << "Withdraw Successful";
} else {
    cout << "Insufficient Balance";
}
```

🔥 Interview Tips

- `switch` faster than multiple `if-else`
- `=` vs `==` common mistake
- Prefer `if-else` for range checking
- Use ternary only for simple logic

🟢 EASY (Basic if / if-else)

1️⃣ Positive / Negative / Zero

একটা integer ইনপুট নাও

- Positive হলে → `"Positive"`
- Negative হলে → `"Negative"`
- Zero হলে → `"Zero"`

2️⃣ Even or Odd

একটা সংখ্যা ইনপুট নিয়ে বলো

- Even
- Odd

3️⃣ Maximum of Two Numbers

দুটি সংখ্যা ইনপুট নিয়ে বড় সংখ্যাটি প্রিন্ট করো।

4️⃣ Pass or Fail

একটা marks ইনপুট নাও

- marks ≥ 40 → `"Pass"`
- না হলে → `"Fail"`

5️⃣ Leap Year Check

একটা year ইনপুট নাও
Leap year কিনা চেক করো।

📌 Rule:

- 400 দিয়ে divisible
- অথবা (4 দিয়ে divisible কিন্তু 100 দিয়ে নয়)

🟡 MEDIUM (else-if, nested if)

6️⃣ Grade Calculator

Marks ইনপুট নাও (0–100)

Marks	Grade
80–100	A+
70–79	A
60–69	B
50–59	C
< 50	F

7️⃣ Largest of Three Numbers

তিনটি সংখ্যা ইনপুট নিয়ে সবচেয়ে বড়টা বের করো।

8️⃣ Voting Eligibility

ইনপুট:

- age
- citizen (0/1)

Condition:

- age ≥ 18 **AND** citizen = 1 → `"Eligible"`
- না হলে `"Not Eligible"`

9️⃣ Simple Calculator (switch)

ইনপুট:

- a, b
- operator (`+ - * /`)

Output:

- result

🔟 Day Name using switch

ইনপুট: 1–7
Output:

- Monday – Sunday
  Invalid হলে `"Invalid day"`

🔴 HARD (Logic + multiple conditions)

1️⃣1️⃣ Electricity Bill Calculator

Units ইনপুট নাও

Units	Charge
0–100	5/unit
101–200	7/unit
>200	10/unit

Total bill বের করো।

1️⃣2️⃣ ATM Simulation

ইনপুট:

- balance
- withdraw amount

Rules:

- withdraw ≤ balance
- withdraw % 500 == 0

না মিললে proper message দেখাও।

1️⃣3️⃣ Triangle Type Checker

৩টা side ইনপুট নাও
Check করো:

- Equilateral
- Isosceles
- Scalene
- Invalid triangle

📌 Hint:

```
a + b > c
```

1️⃣4️⃣ Login System

Input:

- username
- password

Conditions:

- correct → `"Login Successful"`
- wrong username → `"Invalid Username"`
- wrong password → `"Invalid Password"`

1️⃣5️⃣ Ternary Operator Only

একটা সংখ্যা ইনপুট নিয়ে

- Positive / Negative
  শুধু ternary ব্যবহার করে

🧠 Challenge (Interview Level)

1️⃣6️⃣ Scholarship Eligibility

Input:

- GPA

- family income

Rules:

- GPA ≥ 3.5 AND income < 50000 → `"Full Scholarship"`
- GPA ≥ 3.0 → `"Partial Scholarship"`
- Otherwise → `"Not Eligible"`

🎯 Practice Advice

✔ আগে নিজে code লেখো
✔ Test multiple inputs
✔ `if-else` vs `switch` বুঝে ব্যবহার করো

🔁 C++ LOOPS – COMPLETE GUIDE

1️⃣ `for` Loop

🔹 What

যখন **কতবার loop চলবে আগে থেকেই জানা থাকে।**

🔹 Syntax

```
for (initialization; condition; update) {
    // code
}
```

🔹 Flow

1. initialization (একবার)
2. condition check
3. body execute
4. update
5. আবার condition check…

🔹 Example

```
for (int i = 1; i <= 5; i++) {
    cout << i << " ";
```

```
}
```

🔹 Output

```
1 2 3 4 5
```

🔹 Use-case

- Array traversal
- Fixed range loop
- Pattern printing

2️⃣ `while` Loop

🔹 What

যখন **আগে জানি না কতবার চলবে**, কিন্তু condition জানি।

🔹 Syntax

```
while (condition) {
    // code
}
```

🔹 Example

```
int i = 1;
while (i <= 5) {
    cout << i << " ";
    i++;
}
```

🔹 Risk

⚠️ condition false না হলে → **infinite loop**

3️⃣ `do-while` Loop

🔹 What

কমপক্ষে একবার loop চলবেই, condition পরে check হয়।

🔹 Syntax

```
do {
    // code
} while (condition);
```

🔹 Example

```
int i = 10;

do {
    cout << i;
} while (i < 5);
```

🔹 Output

```
10
```

4️⃣ Range-based `for` Loop (C++11+)

🔹 What

Collection / array iterate করার modern way।

🔹 Syntax

```
for (dataType var : collection) {
    // code
}
```

🔹 Example

```
int arr[] = {10, 20, 30};

for (int x : arr) {
    cout << x << " ";
}
```

5️⃣ Infinite Loop

🔹 for Infinite

```
for (;;) {
    cout << "Loop";
}
```

🔹 while Infinite

```
while (true) {
}
```

6️⃣ Nested Loop (Loop inside Loop)

🔹 Example

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        cout << i << j << " ";
    }
    cout << endl;
}
```

🔹 Use-case

- Matrix
- Pattern
- 2D array

7️⃣ `break` Statement

🔹 What

Loop **immediately stop** করে দেয়।

🔹 Example

```
for (int i = 1; i <= 10; i++) {
    if (i == 5)
        break;
```

```
    cout << i << " ";
}
```

Output:

```
1 2 3 4
```

8️⃣ `continue` Statement

🔹 What

বর্তমান iteration skip করে next iteration এ যায়।

🔹 Example

```
for (int i = 1; i <= 5; i++) {
    if (i == 3)
        continue;
    cout << i << " ";
}
```

Output:

```
1 2 4 5
```

9️⃣ `goto` with Loop (Not Recommended ❌)

```
int i = 1;

loop:
cout << i << " ";
i++;

if (i <= 5)
    goto loop;
```

📌 Interview: **avoid using goto**

🔟 Loop with Multiple Variables

```
for (int i = 1, j = 5; i <= j; i++, j--) {
    cout << i << " " << j << endl;
}
```

1️⃣1️⃣ Loop Control Operators Summary

Keyword	Work
break	Exit loop
continue	Skip iteration
return	Exit function

1️⃣2️⃣ Common Mistakes 🚫

❌ `i = 0` vs `i == 0`
❌ Missing `i++`
❌ Infinite loop
❌ Off-by-one error

1️⃣3️⃣ Loop vs Condition (Interview)

Situation	Best Choice
Fixed times	for
Condition-based	while
At least once	do-while
Collection	range-for

1️⃣4️⃣ Real-World Example

Sum of digits

```
int n = 123, sum = 0;
```

```
while (n > 0) {
    sum += n % 10;
    n /= 10;
}

cout << sum;
```

1️⃣5️⃣ Time Complexity Insight

- Single loop → **O(n)**
- Nested loop → **O(n²)**

🧠 Interview Questions (Quick)

1. `while` vs `do-while` difference?
2. Can `for` loop run without condition?
3. Difference between `break` and `continue`?
4. Infinite loop examples?

🔥 Practice Tasks

1. Print 1–100
2. Reverse a number
3. Factorial
4. Fibonacci series
5. Pattern printing

Build in method of C++

🔧 C++ Built-in Methods (Complete Guide)

C++-এ built-in methods আসে মূলত **Standard Library** থেকে।

1️⃣ Input / Output Functions (`<iostream>`)

🔹 `cin`

Input নেওয়ার জন্য

```
int x;
cin >> x;
```

🔹 `cout`

Output দেখানোর জন্য

```
cout << "Hello";
```

🔹 `endl`

New line + buffer flush

```
cout << x << endl;
```

2️⃣ Math Built-in Functions (`<cmath>`)

```
#include <cmath>
```

Function	Work	Example
`sqrt(x)`	Square root	`sqrt(25)` → 5
`pow(a,b)`	a^b	`pow(2,3)` → 8
`abs(x)`	Absolute	`abs(-5)` → 5
`ceil(x)`	Upper round	`ceil(4.2)` → 5
`floor(x)`	Lower round	`floor(4.9)` → 4
`round(x)`	Nearest	`round(4.6)` → 5
`log(x)`	Natural log	
`log10(x)`	Base-10 log	
`sin(x)`	Sin	
`cos(x)`	Cos	

3️⃣ String Built-in Methods (`<string>`)

```
#include <string>
```

- **`length()` / `size()`**

```
string s = "Hello";
cout << s.length();   // 5
```

- **`append()`**

```
s.append(" World");
```

- **`substr(start, len)`**

```
cout << s.substr(0, 4); // Hell
```

- **`find()`**

```
s.find("lo"); // index
```

- **`erase()`**

```
s.erase(0, 2);
```

- **`compare()`**

```
s1.compare(s2);
```

4️⃣ Character Functions (`<cctype>`)

```
#include <cctype>
```

Function	Work
`isupper()`	Uppercase
`islower()`	Lowercase
`isdigit()`	Digit

Function	Work
`isalpha()`	Alphabet
`toupper()`	Uppercase convert
`tolower()`	Lowercase convert

Example:

```
char c = 'a';
cout << toupper(c); // A
```

5️⃣ Array / Algorithm Functions (`<algorithm>`)

```
#include <algorithm>
```

- **`sort()`**

```
sort(arr, arr+n);
```

- **`reverse()`**

```
reverse(arr, arr+n);
```

- **`max()` / `min()`**

```
max(a, b);
min(a, b);
```

- **`count()`**

```
count(arr, arr+n, 5);
```

- **`find()`**

```
find(arr, arr+n, 10);
```

6️⃣ Vector Built-in Methods (`<vector>`)

```
#include <vector>
```

Method	Work
`push_back()`	Add element
`pop_back()`	Remove last
`size()`	Size
`clear()`	Clear all
`empty()`	Check empty
`front()`	First element
`back()`	Last element

Example:

```
vector<int> v;
v.push_back(10);
cout << v.size();
```

7️⃣ Utility Functions (`<cstdlib>`)

Function	Work
`rand()`	Random
`srand()`	Seed
`exit()`	Exit program

```
srand(time(0));
cout << rand();
```

8️⃣ Time Functions (`<ctime>`)

```
time_t t = time(0);
cout << ctime(&t);
```

9️⃣ Type Conversion Functions

- `stoi`, `stof`, `stod`

```
string s = "123";
int x = stoi(s);
```

🔟 File Handling Built-in Methods (`<fstream>`)

```
#include <fstream>
```

Method	Work
`open()`	Open file
`close()`	Close
`getline()`	Read line

1️⃣1️⃣ Common Interview Built-in Combos

```
sort(v.begin(), v.end());
reverse(s.begin(), s.end());
max_element(arr, arr+n);
min_element(arr, arr+n);
```

1️⃣2️⃣ Real-World Example

```
string s;
getline(cin, s);

transform(s.begin(), s.end(), s.begin(), ::toupper);
cout << s;
```

🧠 Interview Tips

✔ Always mention header file
✔ STL functions > manual code
✔ `algorithm` is powerful
✔ Prefer `vector` over array

🔥 Practice Suggestions

1. Sort array using `sort()`
2. Reverse string using built-in
3. Count vowels using `cctype`
4. Random number game

```
#include<iostream>
using namespace std;

int main(){
int a;
while(cin>>a){
cout<<"You entered: "<<a<<endl;
}
return 0;
}
```

6. setprecision

```
#include<iostream>
#include<iomanip>//%% input output manupliation %%
using namespace std;

int main(){
double a=12.3567894;
cout<<fixed<<setprecision(4)<<a<<endl;
}
```

7. if else

```
#include<iostream>
using namespace std;


int main(){
int a;
```

```
cin>>a;
if (a%2==0){
cout<<"even number"<<endl;
}else{
cout<<"Odd number"<<endl;
}
return 0;
}
```

ternary operator

```
%% structure %%
condiatio?true statement: false statement

#include<iostream>
using namespace std;

int main(){
int a=10;
a%2==0 ? cout<<"even number"<<endl : cout<<"Odd number"<<endl;
return 0;
}
```

8. switch

```
#include<iostream>
using namespace std;

int main(){
int val;
cout<<"Enter an integer value: ";
cin>>val;
switch(val%2){
case 0:
cout<<"Even number"<<endl;
break;
case 1:
case -1:
cout<<"Odd number"<<endl;
break;
default:
cout<<"Invalid input"<<endl;
}
return 0;
}
```

9. min,max,swap

```
#include<iostream>
#include<algorithm>
using namespace std;

int main(){
int a,b;
cin>>a>>b;
if (a>b){
cout<<a<<" is greater than " <<b<<endl;
}else{
cout << b << " is greater than " << a << endl;
}
cout<<min(a,b)<<endl;
cout<<max(a,b)<<endl;

// swap
int temp=a;
a=b;
b=temp;

cout<<a<<" "<<b<<endl;
swap(a,b);
cout<<a<<" "<<b<<endl;
return 0;
}
```

10. string

```
#include<iostream>
using namespace std;

int main(){
int a;
char s[30];
// cin>>s;// get input without space
// cout<<s<<endl;

// update
// cin.getline(s,30); // get input with space
// cout<<s<<endl;
cin>>a;
cin.ignore();//enter ke ignore kortehobe
cin.getline(s,30);

cout<<a<<" "<<s<<endl;
return 0;
}
```

stirng constructor

```
#include<iostream>
using namespace std;

int main(){
string s;
cin>>s;
cout<<s<<endl;
return 0;
}
```

header file

```
#include<bits/stdc++.h>
```

this header file all header file ke include kore dey. no needed other include

In C++, a **dynamic object** is an object created at **runtime** using the **heap**, with the `new` keyword.
This allows flexible memory usage and longer object lifetime beyond the function scope.

✅ How to Create a Dynamic Object in C++

1️⃣ Basic Dynamic Object

```
class Student {
public:
    int id;
    string name;
};

int main() {
    Student* s = new Student;   // dynamic object

    s->id = 101;
    s->name = "Atiar";

    cout << s->id << " " << s->name << endl;

    delete s;    // free memory
}
```

2️⃣ Dynamic Object with Constructor

```
class Student {
public:
    int id;
    Student(int i) {
        id = i;
    }
};

int main() {
    Student* s = new Student(10);   // calling constructor

    cout << s->id << endl;

    delete s;
}
```

3️⃣ Dynamic Array of Objects

```
class Student {
public:
    int id;
};

int main() {
    Student* arr = new Student[3];   // array of 3 objects

    arr[0].id = 1;
    arr[1].id = 2;
    arr[2].id = 3;

    delete[] arr;    // free array memory
}
```

4️⃣ Dynamic Object Using Pointer to Access Members

Use `->` operator:

```
Employee* e = new Employee();
e->salary = 50000;
e->work();
```

5️⃣ Dynamic Object with `new` and `delete`

Important Rules:

- Use **new** to allocate
- Use **delete** to deallocate
- For arrays → **delete[]**

6️⃣ Dynamic Objects Using Smart Pointers (Recommended in Modern C++)

Avoid manual memory leaks using `std::unique_ptr`:

```
#include <memory>

unique_ptr<Student> s = make_unique<Student>();
s->id = 100;
```

No need to use `delete` — automatic cleanup.

🟦 Summary

Type	Example
Single dynamic object	`Student* s = new Student();`
Constructor with params	`new Student(10)`
Array of objects	`new Student[n]`
Auto-managed	`make_unique<Student>()`